

Technical Assessment — Smart Cab Dispatch System

Technical Assessment: Smart Cab / Vehicle Dispatch System



Role: Software Engineer · **Submit Before:** 30 July 2026 · **Submission:** [Google Form](#)

Overview

Large group events — conferences, corporate offsites, multi-day gatherings, or any occasion bringing many attendees together — often require coordinating vehicle logistics: picking guests up from airports and railway stations, dropping them at one or more accommodations, and moving them between accommodations and the event venue over the course of the event. Manually coordinating this — matching drivers to guests, tracking arrivals, managing peak-hour surges, and avoiding both guest waiting time and idle vehicles — is error-prone and doesn't scale beyond a handful of people.

This assignment asks you to design and build a mobile application system that automates this coordination: assigning available drivers to guests intelligently, in real time, based on location, timing, vehicle capacity, and live traffic conditions — similar in spirit to how ride-hailing apps work, but adapted for the fixed, scheduled, multi-stop nature of a private group event.

Assignment Objective

Build a mobile-first dispatch system with **two applications** — a **Guest app** and an **Admin Portal** (used by both operations staff and drivers, via separate roles) — that together ensure:

- No guest waits an unreasonable amount of time for pickup.
- No driver sits idle while guests are waiting.
- Vehicle capacity (seats, luggage) is respected and used efficiently.
- Multiple destinations (venue, multiple accommodations, airport, railway station) are handled correctly depending on the point in the event's schedule.
- Assignment happens automatically — neither the guest nor the driver manually browses or picks a match.

Scope of Work

In scope

- Guest mobile app
- Admin Portal (mobile or web) supporting two roles: **Admin/Operations** and **Driver**
- Automated matching/dispatch engine (backend)
- Real-time location tracking and live ETA using a maps/routing API
- Handling of multiple trip types across an event's timeline: airport/station → accommodation (arrival), accommodation → venue (event day), venue → accommodation (return), accommodation → airport/station (departure)

Out of Scope

- Payment processing
- Public marketplace / open driver sign-up (this is a private, event-scoped fleet, pre-registered drivers only)
- Multi-event / multi-tenant support (design for a single event only)

Applications & Roles

1. Guest App — used by invited attendees to view pickup details, get matched, and track their ride.

2. Admin Portal — a single application with two roles:

Role	Description
Admin/Operations	Event-day coordinator with full visibility across all drivers and guests; approves/declines on-demand requests; can manually override assignments.
Driver	Pre-registered driver logs into the same portal under the driver role, sees only their own assigned trip, updates trip status, and shares live location — no visibility into other drivers or the full guest queue.

Keeping drivers inside the Admin Portal (rather than a separate app) simplifies the build — one codebase serves both roles with role-based views/permissions, while the Guest app remains a separate, guest-only experience.

Functional Requirements

1. Guest App

- Guest can register/view their pickup details (flight/train ETA, pickup point, accommodation assigned).
- Guest receives automatic notification once matched with a driver: driver name, vehicle number, live ETA — no option to browse or choose a driver.
- Guest can view live driver location and ETA on a map.
- On arrival, a guest without a pre-scheduled pickup (or wanting an on-demand trip) can raise a ride request from the app. This request goes to the Admin Portal for review, not straight to auto-allocation.
- Guest sees a “request pending” state until admin approves it; once approved, the automated matching engine allocates a driver exactly as it does for scheduled pickups (guest still does not pick the driver).

2. Admin Portal — Admin/Operations role

- Real-time view of all drivers (status, location, current trip) and all guests (waiting, assigned, in transit).
- Approve or decline incoming ride requests raised by guests. Approval is a manual, human decision (e.g. verifying the request is legitimate, checking event-specific rules); once approved, the actual driver allocation itself is handled automatically by the matching engine — admin does not manually pick which driver.
- Driver onboarding is manual: admin/ops staff manually add each driver into the system before the event (name, vehicle number, seat/luggage capacity, contact) — there is no automated driver sign-up or self-registration flow for this assignment’s scope.
- Guest travel/attendance logging is manual where needed: if a guest’s actual travel details deviate from what was pre-registered (e.g. a walk-in guest, a change of flight not reflected in the system), admin/ops staff manually update the guest’s record — the system does not auto-detect or auto-import this.
- Manual override capability for edge cases (priority guest, vehicle breakdown, no feasible auto-match).
- Visibility into upcoming pre-assigned trips and any pending/unmatched guests.

3. Admin Portal — Driver role

- Driver logs in under their own account/role within the same portal app.
- Driver views their assigned trip one at a time (pickup point, guest name/count, destination, ETA target) — no browsing or picking among waiting guests, and no visibility into other drivers’ trips or the admin-level dashboard.
- Driver can accept or reject the specific assigned trip (reject re-queues the guest for reassignment).

- Driver updates trip status (arrived at pickup, guest boarded, arrived at drop) — used to compute halt time and free time for the next assignment.
- Live location shared continuously while on a trip.
- Also consider driver get time to have a proper break between the trip.

4. Matching & Dispatch Engine (backend)

- Pre-day batch assignment for guests with known arrival times.
- Priority-based real-time queue for new/unscheduled requests (including admin-approved on-demand requests), ensuring guests who have waited longer are not indefinitely deprioritized.
- Capacity-aware matching — respects each vehicle's seat and luggage limits; groups larger than any single vehicle are handled via split-and-coordinate or fleet escalation.
- Support for multiple simultaneous destinations (venue and multiple accommodations), clustering same-destination guests for shared rides where appropriate.
- Opportunistic detour insertion — a driver already en route can pick up/drop an additional guest along the way if the added time and remaining capacity allow it, without breaking any existing guest's schedule. Must apply to trips already in progress (using the driver's live position), not only to trips not yet started.
- Continuous re-optimization as live traffic changes ETAs.
- **Note on automation boundary:** driver-to-guest allocation (deciding *which* driver serves *which* guest, and when) is fully automated by this engine. Getting drivers and guests into the system in the first place, and admin's approve/decline decision on ad-hoc requests, are manual steps outside the matching engine's scope.

Non-Functional Requirements

- **Performance:** matching decisions for a new request should resolve in near real time (seconds, not minutes) at the expected scale (10–100 drivers, a few hundred guests).
- **Reliability:** system should degrade gracefully — if the matching engine is briefly unavailable, in-progress trips should not be disrupted, and admin manual override should always remain available.
- **Efficiency:** avoid excessive third-party API usage (e.g. cache static distances, batch requests where possible) to control cost and latency.
- **Usability:** the Guest app and the Driver role within the Admin Portal should require minimal onboarding — assume non-technical users on both sides.
- **Role separation:** the Admin Portal must enforce that a Driver-role user only ever sees their own trip data, never the full operational view reserved for Admin/Operations.

Technical Guidance

- Live routing/traffic data via a maps/routing API (e.g. Google Maps Directions and Distance Matrix APIs)
- A matching approach suited to the problem's scale — a full optimization solver (e.g. Hungarian algorithm / Google OR-Tools) for scheduled batch rounds, and a lightweight greedy match for one-off requests between batches
- Push notifications for guest and driver-role updates
- Role-based access control (RBAC) within the Admin Portal to separate Admin and Driver capabilities
- A backend data model tracking driver state (location, status, vehicle capacity, predicted free time/location) and guest state (pickup/drop, trip type, deadline, group size)

Expected Deliverables

1. Guest app (mobile)
2. Admin Portal (mobile or web) with Admin and Driver roles
3. Backend service implementing the matching/dispatch logic
4. Brief design document explaining your matching algorithm's approach and key trade-offs made

Evaluation & Scoring Criteria

- Correctness and robustness of the matching logic (capacity respected, no guest ignored/starved, deadlines honored)
- Real-world usability of the Guest app and both roles within the Admin Portal
- Clean role separation between Admin and Driver within the single portal
- Handling of edge cases: no feasible driver, mid-trip detour opportunities, multiple accommodations, live traffic changes
- Code quality, architecture clarity, and efficient use of external APIs
- Overall responsiveness of the system under a simulated peak-arrival scenario



How to Submit

1. Push your code to a GitHub repository (Public Repo).
2. Include a with: local setup instructions, architecture overview, matching algorithm explanation, and known trade-offs or limitations.
3. Submit via [Google Form](#)